Constrained EML Grammars: Search Geometry, Rejection Dynamics, and Structural Inflation in Symbolic Regression

Biswajyoti Nath

Independent Study, https://github.com/biswajyoti-nath/eml-framework

Abstract. This paper studies how constrained operator grammars reshape symbolic search behavior in evolutionary symbolic regression. We analyze a grammar built from a single nonlinear primitive, $\mathrm{eml}(x, y) = \exp(x) - \ln(y)$, under strict real-domain positivity constraints with conservative symbolic validity enforcement, grammar-pure mutation without repair, and rejection-aware accounting in which every candidate attempt—accepted or rejected—consumes evaluation budget. Across a structural benchmark study and an exploratory evolutionary experiment on 18 domain-safe targets, we observe three coupled effects. First, recursive EML macro-expansion systematically inflates symbolic tree depth and node count relative to native exp/log representations. Second, the constrained grammar induces substantially sparser valid search regions, increasing rejection density under strict validity enforcement. Third, elevated rejection density reduces effective evolutionary throughput, so identical nominal budgets support fewer productive generations. Rejection decomposition further shows that throughput loss is dominated primarily by structural depth overflow arising from recursive macro-expansion rather than by numerical instability alone. These findings are presented as a controlled empirical characterization of validity-aware symbolic search under operator-space restriction, not as a claim of algorithmic superiority.

Keywords: symbolic regression · operator-space constraints · structural inflation · rejection rate · EML representation

1 Introduction

In symbolic regression, the choice of grammar is not merely syntactic—it determines which expressions are directly reachable, which require multi-step encoding, and which are effectively excluded by domain constraints. Restricting the operator set therefore reshapes the geometry of symbolic search: the density of valid candidates within the constrained manifold, the depth of minimal encodings, and the fraction of the evaluation budget consumed by rejected individuals all change as a function of the grammar. Highly constrained grammars provide a controlled setting for studying how operator-space restrictions reshape valid symbolic search regions, allowing observation of representation-induced search distortion under strict validity enforcement.

This work studies one such restriction: replacing the standard exp/log operator set with a single nonlinear primitive, the EML operator,

$$\mathrm{eml}(x, y) = \exp(x) - \ln(y), \tag{1}$$

which has been shown to generate all elementary functions over the complex field when composed with the constant 1 [1]. Our scope is narrower and empirical. We ask: when this operator restriction is enforced in a real-valued system—where logarithm arguments must be strictly positive, complex-field identities are unavailable, and every rejected candidate consumes evaluation budget—how does the resulting search behavior differ from that of the standard grammar?

The investigation makes no claim about universal expressivity or practical algorithmic superiority. It is a controlled characterization of representation-induced search distortion—how a constrained operator grammar reshapes symbolic structure and interacts with evolutionary throughput under strict validity constraints. The contributions are: (1) a precise operational definition of the supported EML fragment under real-domain constraints; (2) a reproducible experiment design with true rejection-aware budget accounting that explicitly separates domain, depth, and numeric rejection types; and (3) empirical evidence that structural inflation, rejection density, and throughput reduction emerge naturally from this operator restriction.

The remainder of the paper proceeds as follows. Section 2 defines the EML representation and its domain constraints. Section 3 describes the experimental design. Sections 4 and 5 report structural and regression results, respectively. Section 6 discusses limitations and implications.

2 Background and Representation

2.1 Related Work in Constrained Search

The interaction between grammar constraints and search geometry has long been recognized in genetic programming. Montana's Strongly Typed GP [6] established that restricting the operator space via type constraints can drastically reduce the search space, though it often requires complex initialization to find valid trees. Grammar-guided GP [7,9] extended these ideas by using formal grammars to constrain program structure, demonstrating that production rules dictate the topology of the candidate manifold. Subsequent work in constrained symbolic regression [8] and semantic constraints [10] has explored how domain restrictions reshape the distribution of valid candidates. Operator-restricted search has also been studied in the context of parsimony and interpretability [11]. This paper builds on these foundations by analyzing the operational throughput cost of strict domain constraints when enforcing a single-operator grammar, extending the analysis of representation-induced search distortion to include explicit rejection-type accounting.

2.2 The EML Fragment

The representational choices described in this section determine both the structural properties of EML expressions and the domain constraints that govern candidate validity in the search experiments that follow.

The EML operator is a binary primitive with mixed exponential-logarithmic semantics. Under real-valued computation, the second argument of $\mathrm{eml}(x, y)$ must remain strictly positive so that $\ln(y)$ is well defined. The implementation treats the following identities as the basis for EML encoding:

$$\exp(x) = \mathrm{eml}(x, 1), \tag{2}$$

$$\ln(x) = 1 - \mathrm{eml}(0, x), \tag{3}$$

where the second identity requires $x > 0$ in the real domain. These rewrite rules are the source of the structural inflation analyzed in later sections: each unary transcendental node expands into a multi-node EML subexpression. Because arithmetic absorption requires recursive macro-expansion, structural growth is expected to increase systematically relative to native exp/log representations.

More complex operations are absorbed into the EML primitive whenever positive-domain assumptions permit. Multiplication can be encoded as

$$a \cdot b = \exp\big(\ln(a) + \ln(b)\big), \tag{4}$$

and non-integer powers as

$$x^a = \exp\big(a \cdot \ln(x)\big) \quad (x > 0). \tag{5}$$

Division can be encoded as

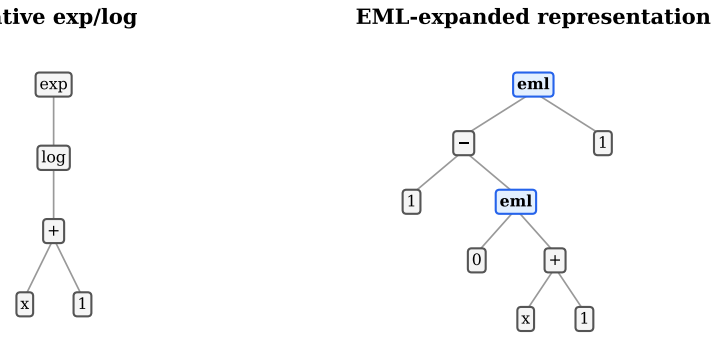
$$x/y = \exp\big(\ln(x) - \ln(y)\big), \tag{6}$$

which expands to $\mathrm{eml}(\ln(x) - \ln(y), 1)$ under EML. Square roots follow as

$$\sqrt{x} = \exp\big(\frac{1}{2} \cdot \ln(x)\big) \quad (x > 0), \tag{7}$$

and more complex operations compound this inflation. For instance, the expression $\sqrt{x}/(x+1)$ requires encoding as $\exp(\frac{1}{2}\ln(x) - \ln(x+1))$, which in EML form becomes $\mathrm{eml}(\frac{1}{2}\ln(x) - \ln(x+1), 1)$, where each ln term itself expands into $1 - \mathrm{eml}(0, \cdot)$. These examples illustrate how even simple algebraic operations balloon into deeply nested EML structures, making the inflation visceral: what appears as a compact expression in standard notation becomes a sprawling tree under the constrained grammar.

Symbolic positivity checking is conservative by design. The domain tests return false unless SymPy can prove positivity under its assumption system, so symbolic certifiability is a stricter condition than mathematical positivity on a restricted real interval. This conservatism increases rejection density but ensures that every accepted candidate is semantically valid.



Representation	Nodes	Depth
Native exp/log	4	3
EML-expanded	7	4

Fig. 1: Side-by-side comparison of $\exp(\ln(x+1))$ under native exp/log representation (left) and EML-expanded representation (right). The EML encoding replaces native nonlinear composition with recursive primitive expansion. This inflation emerges directly from the rewrite rules and scales with expression complexity.

2.3 Structural Inflation and Grammar Purity

Structural inflation denotes the systematic increase in node count and tree depth that results from rewriting expressions into the EML fragment. This is not an implementation artifact; it is an inherent consequence of encoding multiple nonlinear operators through a single binary primitive. Each $\exp(x)$ node expands into $\mathrm{eml}(x,1)$, and each $\ln(x)$ becomes $1-\mathrm{eml}(0,x)$—adding nesting and auxiliary nodes even for elementary transcendental subexpressions.

Because arithmetic absorption requires recursive macro-expansion, structural growth is expected to increase systematically relative to native exp/log representations. The encoding rules create a predictable expansion pattern: unary transcendental operators expand into multi-node EML subexpressions, and compositions cascade this expansion. For example, a simple product $a \cdot b$ requires encoding as $\exp(\ln(a)+\ln(b))$, which in EML form becomes $\mathrm{eml}(\ln(a)+\ln(b),1)$, where each ln term itself expands. This compounding effect means that expressions with multiplicative structure or nested transcendental compositions experience disproportionately higher inflation. Figure 1 illustrates this expansion concretely for the expression $\exp(\ln(x+1))$.

To isolate the effect of operator restriction, the implementation enforces *grammar purity*: EML mutation operates strictly within the EML grammar, with eml as the sole nonlinear primitive and all multiplication and power operations

expressed via EML identities. The baseline grammar uses native SymPy operators (exp, ln, multiplication, integer power) without restriction.

2.4 Rejection Dynamics and Evaluation Budget

The study adopts a strict operational definition of budget: **every attempted candidate consumes one evaluation**, whether it is discarded by a depth gate, rejected for domain invalidity, or accepted for fitness scoring. This true budget accounting prevents the analysis from masking invalid sampling behind a nominal budget cap.

Rejection rate is treated as a first-class characterization metric, not a nuisance variable to normalize away. The causal chain is direct: higher rejection density indicates sparser valid regions within the sampled grammar; sparser valid regions mean that a larger fraction of the evaluation budget is consumed by non-productive candidates; and this in turn reduces effective search throughput—the number of productive generations the search can complete under a fixed budget. This relationship between rejection density and effective throughput is central to the results that follow.

3 Experimental Design

The structural and search-behavior differences identified in Section 2 motivate a controlled experiment. The search procedure is intentionally simple in order to isolate representational effects from optimizer-specific heuristics. This is a controlled grammar study, not a competitive optimizer evaluation; advanced archiving, age-fitness Pareto optimization, or complex repair mechanisms are explicitly excluded to prevent them from masking the raw structural impact of the representation. The intentional simplicity ensures that observed differences are attributable to grammar-induced search-space geometry rather than optimizer-specific enhancements. The search runs with the following fixed parameters:

- Population size: 100.
- Elite size: 15.
- Total evaluation budget: 4,000 candidate evaluations per run.
- Maximum tree depth: 6.
- Trials: 5 independent runs with deterministic seeds.

The depth cap is set to preserve nominal parity between grammars while exposing the additional structural cost of EML macro-expansion. Crucially, the budget is shared across grammars: every rejected evaluation reduces the remaining budget for productive search.

For each grammar, the search proceeds as follows:

1. Generate an initial valid population via rejection sampling.
2. Score the population with strict numeric validity checks.
3. Retain elites and generate offspring via grammar-pure mutation.
4. Reject any candidate whose actual tree depth exceeds the depth limit.

Table 1: Structural inflation and rejection dynamics across 18 targets. Values show mean ± std across 5 independent trials per target. Rejection rates (%) are disaggregated by type: domain (symbolic constraint violations), depth (structural limits), numeric (evaluation failures), and total. Total rejection is shown as an aggregate percentage and is not accompanied by a standard deviation because it is derived from the category-level totals. Depth rejection dominates under EML (75.2±0.7% vs. 6.6±6.0% baseline) with notably lower variance, indicating consistent structural capacity constraints across targets.

Grammar	Nodes	Depth	Domain (%)	Depth (%)	Numeric (%)	Total (%)	Gens
exp/log baseline	10.6±4.0	3.2±0.9	6.5±5.0	6.6±6.0	31.5±9.7	44.6	15.5±1.0
EML grammar	14.8±5.2	4.4±0.9	0.6±0.8	75.2±0.7	10.0±1.5	85.9	5.1±0.4
Ratio (EML/bl)	1.4×	1.4×	0.1×	11.4×	0.3×	1.9×	0.3×

5. Count every candidate attempt—accepted or rejected—toward the evaluation budget.

The EML grammar uses a dedicated random generator and mutation operators that operate strictly within the EML fragment. The baseline grammar uses standard SymPy tree construction. There are no repair mechanisms, no fallback fitness assignments, and no hidden equivalence testing.

Rejection is not treated as a monolithic failure. To clarify the causal mechanics of search distortion, we explicitly define and separate three rejection types:

- **Domain Rejection:** Candidates rejected during symbolic verification because their structure violates mathematical domain constraints (e.g., negative arguments to a logarithm).
- **Depth Rejection:** Candidates rejected because their structure exceeds the strict maximum depth cap.
- **Numeric Rejection:** Candidates rejected during fitness evaluation due to numerical overflow, NaN generation, or floating-point errors on the sample grid.

This separation prevents the conflation of syntactic depth limits with semantic domain violations, allowing future iterations of this study to report detailed rejection profiles for each grammar.

The regression targets comprise 18 domain-safe functions selected from a larger expression catalogue. Each target is evaluated on a fixed sample grid of 201 points within a positive-domain interval. All experiments are deterministic given the seed settings in `src/eml/config.py`.

4 Structural Characterization Results

The structural analysis quantifies the representational cost of operator restriction. Table 1 aggregates the key metrics across all 18 targets.

The EML grammar produces trees with 1.4× more nodes and 1.4× greater depth on average, while experiencing 1.9× higher aggregate rejection rates (85.9%

vs. 44.6%). Consequently, EML completes only 0.3× as many generations as the baseline under the same evaluation budget.

The distribution of rejection rates across targets further illustrates the search-space sparsity difference between grammars. Figure 2 shows that EML rejection rates cluster tightly between 71% and 79%, while baseline rates span a wider range from 18% to 45%. This indicates that the valid candidate region under EML is not only sparser on average but also consistently sparse across diverse target structures.

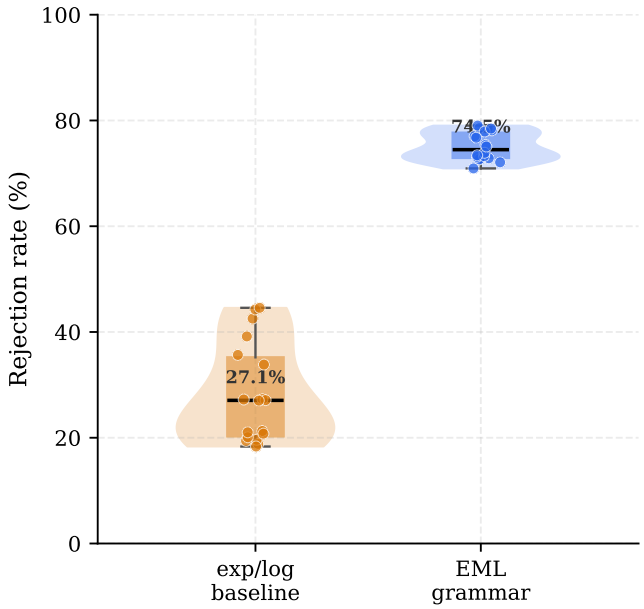


Fig. 2: Boxplot comparison of aggregate rejection rates across 18 targets under EML and baseline grammars. EML exhibits consistently higher rejection rates (median approximately 75%) compared to the baseline (median approximately 27%). These aggregate rates include domain, depth, and numeric rejections under strict validity enforcement.

The mechanism underlying these differences is traceable to the rewrite rules. Each $\exp(x)$ becomes $\mathrm{eml}(x, 1)$ and each $\ln(x)$ becomes $1 - \mathrm{eml}(0, x)$, adding nesting and auxiliary nodes for every transcendental subexpression. Multiplicative encodings such as $a \cdot b = \exp(\ln(a) + \ln(b))$ compound this effect when the operands are themselves nontrivial. Structural inflation is therefore not an artifact to be normalized away; it is an inherent representational cost of the constrained grammar.

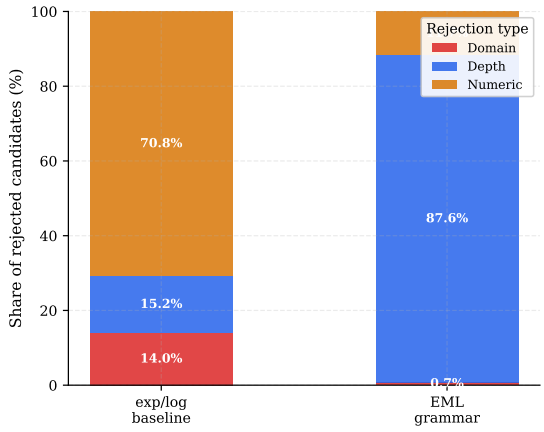


Fig. 3: Composition of rejected candidates by type. Percentages are expressed relative to the total number of rejected candidates, so the bars sum to 100% within each grammar. Under EML, depth-triggered rejections constitute 87.6% of all rejections, reflecting structural inflation as the dominant throughput constraint. Domain rejections comprise only 0.7% of EML rejections, while numeric rejections account for 11.7%. The baseline grammar shows a more balanced composition: numeric rejections (70.8%) predominate, with depth (15.2%) and domain (14.0%) rejections contributing more equally.

The disaggregated rejection metrics reveal that depth rejection dominates under EML ($75.2 \pm 0.7\%$ vs. $6.6 \pm 6.0\%$ baseline), as structural inflation exhausts the depth cap before domain verification. Domain and numeric rejections are correspondingly lower. Figure 3 shows EML rejections are 87.6% depth-related, versus a more balanced distribution in the baseline.

5 Symbolic Regression Results

The structural patterns documented above have direct consequences for evolutionary search. The relationship between rejection density and effective throughput is shown in Figure 4, which plots rejection rate against productive generations completed for each target under both grammars.

This separation is clear: EML targets cluster in the high-rejection, low-generation region, while baseline targets occupy the opposite quadrant, confirming that higher rejection density reduces productive generations under fixed budgets.

The exploratory regression experiment produced additional characterization observations:

– Below-threshold MSE (indicating functional recovery) was observed on 8 of 18 targets under the EML grammar and 6 of 18 under the baseline. These counts are reported as search-behavior indicators, not as competitive performance metrics.

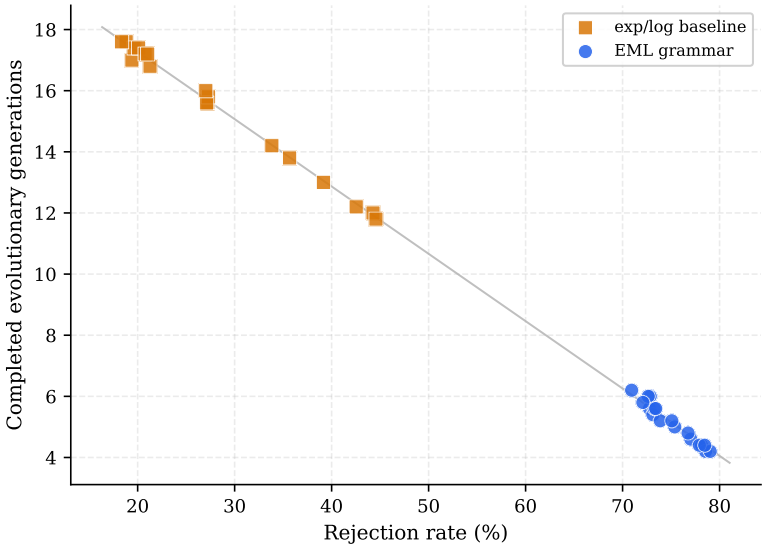


Fig. 4: Operational consequence of rejection-aware budget accounting: higher aggregate rejection density reduces the number of productive generations achievable under a fixed evaluation budget of 4,000 candidates. This clustering pattern is an expected outcome of the experimental design—as rejection rates increase, proportionally fewer evaluations remain for productive evolutionary updates—not a claim of discovering a novel statistical law.

These observations characterize how each grammar interacts with the search process; they do not constitute evidence that either grammar is categorically preferable. Some targets remained recoverable under EML despite severe throughput reduction. Targets exhibiting recursive exp/log compatibility—pure exponentials ($\exp(x)$, $\exp(x^2)$), pure logarithms ($\ln(x)$), and nested compositions ($\exp(\ln(x+1))$, $\ln(\exp(x))$)—were consistently recovered, as these structures align directly with the EML primitive's encoding pattern. Notably, several structurally aligned targets (e.g., $\exp(x^2)$, $\exp(x)+x$) were not recovered within the evaluation budget by the baseline grammar, illustrating how constrained recoverability depends on representational alignment rather than raw operator diversity. Conversely, mixed multiplicative targets ($x^2 \cdot \ln(x)$, $x^2 \cdot \exp(x)$) and rational forms ($x/(1+x)$) incurred deeper macro-expansions that compounded rejection penalties under EML. This pattern suggests that recoverability is governed by structural affinity to the constrained grammar's encoding manifold, with both grammars exhibiting target-dependent recoverability profiles.

The central observation is that equal nominal budgets do not imply equal effective search capacity. The same 4,000 candidate evaluations yield fewer productive generations for EML because a larger fraction of the budget is consumed

by rejected candidates and by the deeper structures that surviving candidates require.

6 Discussion and Limitations

The results reveal three interrelated consequences of operator restriction that merit further interpretation.

Conservative Validity Enforcement The EML transformer rejects expressions unless SymPy can prove all logarithm arguments and non-integer power bases are positive. Symbolic certifiability is therefore a stricter condition than mathematical positivity on a restricted domain: expressions that are numerically valid on the sampled interval may still fail the symbolic check. This conservatism increases rejection density but prevents semantically invalid candidates from entering the population.

Operator Restriction Reduces Valid Search-Space Density Replacing multiple nonlinear primitives with a single binary operator simplifies the grammar in terms of operator variety, but it concentrates valid expressions into a sparser subregion of the candidate space. The elevated rejection rates observed under EML are not transient search inefficiency; they reflect a structurally sparser valid region within the constrained grammar's candidate manifold.

Structural Inflation Interacts with Depth Constraints The EML grammar can encode many functions in principle, but doing so requires systematically deeper and wider trees. Under a strict depth cap, this inflation reduces the reachable portion of the search space. The resulting representation-induced search distortion—where structurally deeper encodings consume depth budget that would otherwise support broader exploration—is an inherent property of the constrained grammar, not a quantity to be normalized away.

Rejection Type Analysis Clarifies Operational Mechanisms The disaggregated rejection metrics enable a more nuanced interpretation of search behavior. Depth rejection dominates under EML ($75.2 \pm 0.7\%$ vs. $6.6 \pm 6.0\%$ baseline), indicating that structural inflation operationalizes the primary throughput constraint—the grammar's encoding of domain-safe representations consumes depth budget at scale. Domain rejection is lower under EML (0.6% vs. 6.5% baseline) because depth filtering occurs first, removing structurally excessive candidates before domain verification. Numeric rejection is also reduced under EML (10.0% vs. 31.5% baseline) as a secondary effect of depth filtering. This cascade—structural inflation (required for domain-safe encodings) triggers depth rejection, which subsequently constrains domain verification—reveals that EML's fundamental limitation is representational compactness under domain

constraints, not semantic validity in isolation. The constrained grammar produces valid expressions when depth permits; the challenge is that permitted depths are systematically shallower due to the macro-expansion required for domain safety.

The study has clear limitations, reported here to bound the conclusions that can be drawn. The benchmark suite is small and restricted to functions expressible within the positive-domain exp–log fragment. The evolutionary search strategy is deliberately simple; it is not designed for state-of-the-art symbolic regression and should not be evaluated as such. The EML fragment excludes trigonometric, hyperbolic, and complex-valued constructions, so the results do not address the full theoretical expressivity of the EML operator. Conservative positivity checking may reject expressions that are provably safe under more sophisticated domain analysis. No claim is made that EML is preferable to standard grammars for any specific application.

7 Conclusion

This study has characterized how a constrained operator grammar—built from the single nonlinear primitive $\mathrm{eml}(x, y) = \exp(x) - \ln(y)$—reshapes symbolic search behavior under real-domain constraints. Three findings emerge consistently from the experiments.

First, operator restriction induces systematic structural inflation: each transcendental subexpression expands into a deeper, wider EML encoding, increasing both node count and tree depth across all targets. Second, domain constraints—particularly the strict positivity requirements on logarithm arguments—produce elevated rejection rates that reflect a sparser valid region within the constrained grammar's candidate space. Third, the combination of higher rejection density and deeper candidate structures reduces effective search throughput: the same nominal evaluation budget supports fewer productive generations under EML than under the standard grammar.

These three effects—structural inflation, sparse validity, and throughput reduction—are not independent. They emerge jointly from the decision to restrict the nonlinear operator set. Under fixed evaluation budgets, increased rejection density reduces the number of productive evolutionary updates available to the constrained grammar. This study demonstrates that rejection-aware budget accounting reveals the cost of invalid candidate sampling under constrained grammars: every rejected evaluation consumes budget that would otherwise support productive search. Rejection-aware budget accounting is therefore a fundamental consideration in constrained symbolic regression design, not a secondary implementation detail, and practitioners should consider strategies such as adaptive depth caps, rejection-stratified budgets, or grammar-aware population sizing when operating under strict validity constraints. Characterizing these dynamics under controlled conditions is a necessary step toward principled grammar design in domain-restricted symbolic search.

References

1. A. Odrzywołek, "All elementary functions from a single binary operator," arXiv:2603.21852, 2026.
2. S. M. Udrescu and M. Tegmark, "AI Feynman: A physics-inspired method for symbolic regression," *Science Advances*, vol. 5, no. 3, 2019.
3. K. Petersen, J. Zhou, A. Nelatury, and D. Carmel, "Deep symbolic regression," in *Proceedings of the International Conference on Learning Representations*, 2021.
4. W. La Cava, J. H. Moore, and R. J. Urbanowicz, "FEAT: Feature engineering automation tool for predictive modeling," *Journal of Machine Learning Research*, vol. 21, no. 150, 2020.
5. M. Schmidt and H. Lipson, "Distilling free-form natural laws from experimental data," *Science*, vol. 324, no. 5923, pp. 81–85, 2009.
6. D. J. Montana, "Strongly typed genetic programming," *Evolutionary Computation*, vol. 3, no. 2, pp. 199–230, 1995.
7. R. I. McKay, C. Hoai, P. M. Whigham, Y. Shan, and M. O'Neill, "Grammar-based genetic programming: a survey," *Genetic Programming and Evolvable Machines*, vol. 11, pp. 365–396, 2010.
8. Q. Lu, "Constrained symbolic regression," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 46, no. 10, pp. 1345–1356, 2016.
9. P. A. Whigham, "Grammatically-based genetic programming," in *Proceedings of the Workshop on Genetic Programming: From Theory to Real-World Applications*, 1995, pp. 33–41.
10. L. Trujillo, A. Munoz, L. Schoenauer, and M. Secretan, "Optimization for the evolving grammar-based genetic programming," *Genetic Programming and Evolvable Machines*, vol. 17, pp. 483–513, 2016.
11. M. Virgolin, A. Aldén, and T. Alderliesten, "Learning where to look: Neural attention for symbolic regression," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2021, pp. 1635–1643.